Relatório — Compilador Pasquick e IDE Chronus

9 de novembro de 2026

Identificação

Curso	Ciência da Computação
Disciplina	Compiladores
Professor	Rodrigo Freitas Silva
Trabalho	Trabalho Final — Compilador (análises léxica, sintática e semântica)
Aluno(s)	*Gustavo Wesley de Souza, Thiago da Silva Abreu Souza, Luigi Camporez de Oliveira*
Linguagem	Pasquick (arquivos `.pqk`)
IDE	Chronus

Introdução

Este relatório descreve o trabalho final da disciplina de Compiladores: um compilador para um dialeto de Pascal, feito a partir da gramática do Anexo I, e uma pequena IDE para escrever e testar os programas. Na primeira entrega o projeto tinha só o analisador léxico. Agora o programa passa pelas três fases de análise, léxica, sintática e semântica, e os erros de qualquer uma delas aparecem enquanto o código está sendo digitado.

A ideia do começo continuou valendo: fazer algo simples de entender e de explicar. Seguimos com C puro, com a raylib só para a parte gráfica, e sem ferramentas que geram o analisador, como o Flex e o Bison. O autômato do léxico já tinha sido escrito à mão, estado por estado, e o sintático foi pelo mesmo caminho: cada regra da gramática virou uma função em C. O semântico, por sua vez, é um percurso pela árvore que o sintático monta.

As seções sobre o analisador léxico foram mantidas, com os ajustes necessários, porque ele continua sendo a base de tudo. As partes novas são as do analisador sintático, do analisador semântico e as mudanças na IDE.

O que mudou desde a primeira entrega

- Corrigimos o problema apontado na avaliação: o ponto do final do programa não aparecia como token quando havia um comentário sem fechamento antes dele.
- Novo analisador sintático, que confere a estrutura do programa, monta a árvore sintática e continua depois de cada erro.
- Novo analisador semântico, com escopos, tabela de declarações e verificação de tipos.

- Um módulo novo, `compilador.c`, que roda as fases em sequência e permite parar depois de qualquer uma delas.
- Na IDE, duas abas novas (Árvore e Declarações), o botão Fase (F6) e a fase de cada erro na mensagem.
- A versão de console passou a se chamar `pasquick.exe` e ganhou opções para mostrar cada fase separadamente.
- Três programas de teste novos.

A linguagem Pasquick

Batizamos a linguagem de Pasquick, uma mistura de Pascal com *quick*, porque no fundo ela é um Pascal reduzido ao que a gramática pede. A IDE ganhou o nome de Chronus, e os arquivos de código usam a extensão `.pqk`.

Na parte léxica, seguimos o que o enunciado define. Não há diferença entre maiúsculas e minúsculas, então `begin`, `BEGIN` e `Begin` são a mesma palavra. Identificadores começam com letra, depois podem ter letras, dígitos e `_`, e têm no máximo 15 caracteres. Palavras reservadas não podem ser usadas como nome de variável.

Uma particularidade é que os operadores lógicos são em português, `E` e `OU`, do jeito que aparecem na gramática. Isso tem um efeito curioso: não dá para ter uma variável chamada `e`. O `FOR` e o `TO`, que na primeira entrega estavam só reservados, agora são usados pelo comando `for`.

Os números podem ser inteiros (`10`) ou reais com ponto (`3.14`). Os literais ficam entre aspas simples e não podem quebrar linha. Comentários podem ser de uma linha, com `//`, ou de várias, entre `{ }` ou `(* *)`. Tanto os comentários quanto os espaços são descartados pelo analisador.

Um programa começa com `program` e o nome, depois vêm as declarações e por fim um bloco `begin ... end` terminado por ponto. As declarações podem ser de variáveis (`var`), constantes (`const`), procedimentos e funções, em qualquer ordem e quantas vezes for preciso. Os tipos são `integer`, `real`, `char` e `string`, e qualquer um deles vira vetor quando se indica o tamanho entre colchetes, como em `notas : real[5]`.

Os comandos são a atribuição (`:=`), a chamada de procedimento, o bloco `begin ... end`, o `if` com `else` opcional, os laços `while`, `repeat ... until` e `for ... to`, e o `break` e o `continue`, para sair do laço ou pular para a próxima volta. Como em Pascal, uma função devolve seu valor quando se atribui algo ao nome dela.

A gramática não tem comandos de entrada e saída. Para os programas conseguirem mostrar alguma coisa, deixamos quatro procedimentos já declarados: `write`, `writeln`, `read` e `readln`. Eles não são palavras reservadas, são só nomes que o semântico já conhece quando a análise começa.

Um programa pequeno, só para dar uma ideia de como a linguagem fica (o `exemplos/completo.pqk` usa todas as construções):

```
program Exemplo;
var
    x : integer;
begin
    x := 0;
    while x < 10 do
        x := x + 1;
end.
```

Como foi o desenvolvimento

Antes de escrever qualquer código, lemos o enunciado e a gramática com calma, listamos todos os tokens e tentamos prever os casos estranhos. Algumas decisões saíram dessa etapa: literal não pode ocupar mais de uma linha, comentário dentro de comentário não é aceito, e algo como `12abc` é erro, e não um número seguido de um identificador.

A primeira versão do analisador não tinha interface nenhuma. Era um programa de console que lia um arquivo `.pqk` e imprimia os tokens, a tabela de símbolos e os erros. Parece um passo a mais, mas ajudou muito, porque deu para testar o autômato sozinho, sem misturar problema de interface com problema de análise.

Com o lexer funcionando, instalamos a raylib pelo instalador para Windows, que já vem com o gcc, e partimos para a IDE. O editor de texto foi feito do zero. O programa inteiro fica num único vetor de `char`, o cursor é só um índice nesse vetor, e inserir ou apagar um caractere é um `memmove`. É uma solução bem simples, mas para programas do tamanho dos nossos exemplos funciona sem atraso nenhum.

Na primeira parte, duas coisas pediram mais cuidado do que esperávamos. A primeira foram os acentos. O texto é UTF-8, e um caractere como `á` ocupa dois bytes, então se contássemos bytes a coluna dos erros sairia errada em qualquer linha com acento antes. Por isso as colunas são contadas por caractere, ignorando os bytes de continuação, tanto no lexer quanto no editor. A segunda só apareceu quando vimos a IDE rodando: um trecho com erro, como `12abc`, não vira token, e o editor estava pintando ele com a mesma cor dos comentários. Corrigimos pintando esses trechos de vermelho claro, usando a posição guardada em cada erro.

Para a segunda parte, repetimos o roteiro. Primeiro reescrevemos a gramática do Anexo I em EBNF, num formato que desse para passar direto para o código: sem recursão à esquerda e com cada alternativa começando por um token diferente. Depois escrevemos o sintático e o semântico só na versão de console, imprimindo a árvore no terminal. Foi olhando a árvore impressa que conferimos, por exemplo, se em `2 + 3 * 4` a multiplicação ficava embaixo da soma, como deveria. Só com tudo funcionando no console é que ligamos as fases novas à IDE.

Três problemas tomaram mais tempo do que esperávamos. O primeiro foram os erros em cascata: no começo, um único `;` esquecido gerava uma lista de erros nas linhas seguintes, quase todos falsos. A solução foi a recuperação em modo pânico com um indicador que segura novas mensagens até o analisador se reencontrar, como explicamos mais adiante.

O segundo foi a posição dos erros. Quando falta o `;` no fim de uma linha, o analisador só percebe isso ao ler o primeiro token da linha seguinte, e a mensagem acabava apontando para o

lugar errado. Passamos a marcar esse tipo de erro logo depois do último token da linha anterior, que é onde o `;` deveria estar.

O terceiro foi o semântico rodando em cima de uma árvore incompleta. Se uma declaração tinha erro de sintaxe, ela não entrava na árvore, e o semântico acusava "não declarado" em todo lugar onde a variável aparecia. Decidimos que o semântico só roda quando não há erro sintático. Pode parecer uma limitação, mas é o que os compiladores costumam fazer, e evita mostrar erros que na verdade não existem.

As análises não dependem da interface: IDE e console chamam a mesma função

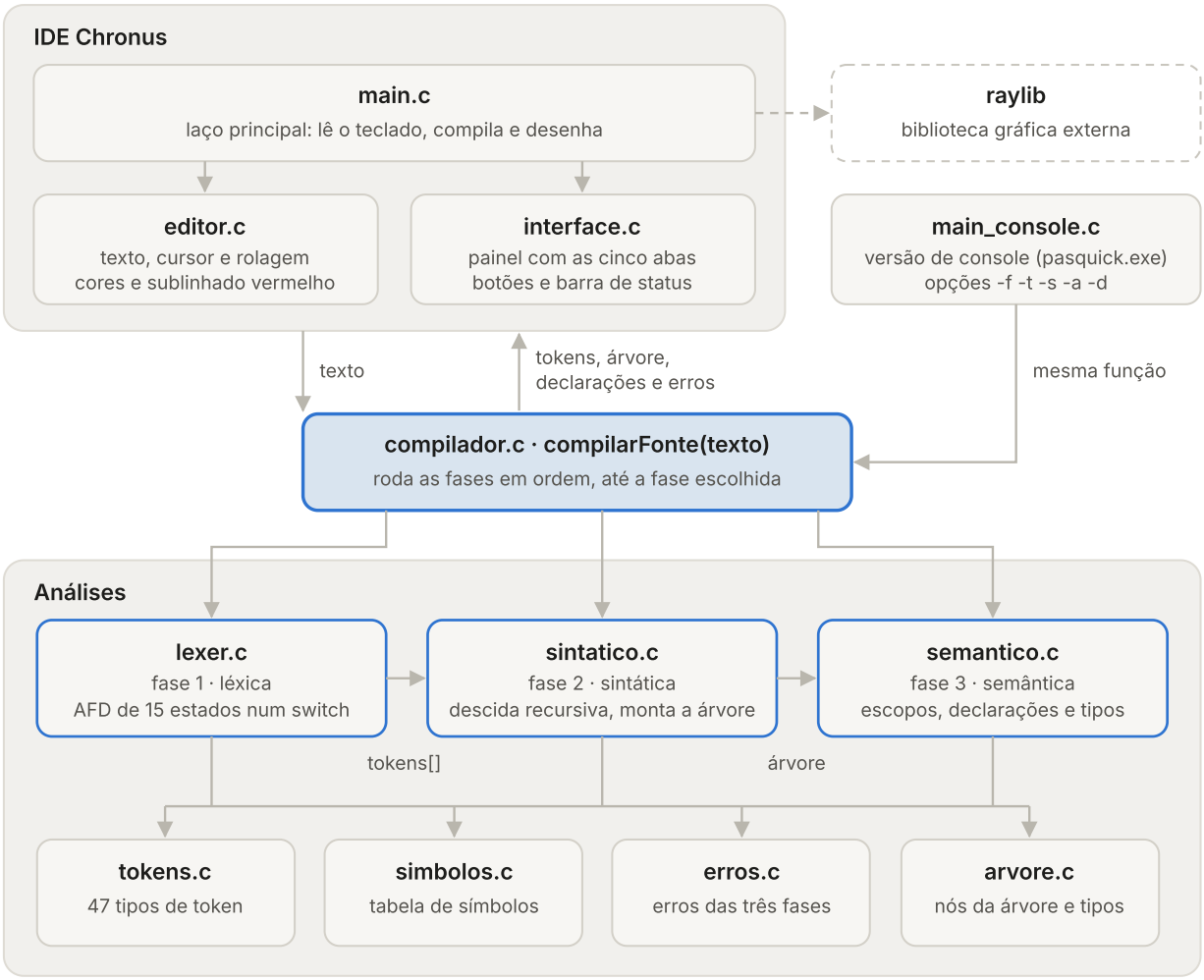


Diagrama dos módulos · 12 arquivos `.c` + raylib. Seta cheia: chamada ou dados. Tracejado: biblioteca externa, usada também por `editor.c` e `interface.c`.

O desenho da primeira entrega continuou valendo: as análises nem sabem que existe uma interface. A diferença é que agora a IDE e o console não chamam mais o lexer diretamente, e sim a função `compilarFonte()`, em `compilador.c`, que roda as fases uma depois da outra. Cada fase lê o resultado da anterior e acrescenta o seu: o léxico gera os tokens, o sintático transforma os tokens numa árvore e o semântico anota tipos nessa árvore e monta a tabela de declarações. Os erros de todas as fases vão para a mesma lista, cada um com a sua fase, e no fim são ordenados por linha e coluna.

A `compilarFonte()` também guarda até qual fase a análise deve ir. É isso que permite mostrar o compilador por partes, como explicamos na seção da IDE.

Analisador léxico

Esta parte é a da primeira entrega, com a correção pedida na avaliação.

Como funciona

Todo o trabalho acontece na função `analisarFonte()`, em `lexer.c`. Ela percorre o texto uma única vez, caractere por caractere, dentro de um laço com um `switch` sobre o estado atual. Cada `case` desse `switch` é um estado do autômato mostrado mais abaixo, então dá para ler o código com o desenho do lado.

No estado inicial, o analisador anota onde o lexema começa e decide para onde ir olhando só o caractere atual. Ao chegar num estado final, ele chama `emitir()`, que guarda o token com tipo, lexema, linha, coluna e atributo.

Um detalhe que simplificou bastante: palavras reservadas e identificadores seguem exatamente o mesmo caminho no autômato. Depois de ler a palavra inteira, o analisador passa ela para maiúsculo e procura na tabela de símbolos. Se ela já está lá como reservada, sai o token correspondente, como `WHILE`. Se não está, a palavra entra na tabela como identificador e sai um `ID`. Sem isso, teríamos que criar estados separados para cada uma das 24 palavras reservadas.

A tabela de símbolos é um vetor de structs com busca linear. Sabemos que não é a estrutura mais eficiente, mas com algumas dezenas de entradas a diferença não aparece, e o código fica fácil de acompanhar. Toda análise começa com 43 entradas fixas (24 palavras reservadas, 11 operadores e 8 sinais de pontuação), e cada identificador novo vai para o fim, junto com a linha onde apareceu pela primeira vez.

Erros e recuperação

Para a recuperação de erros escolhemos o modo pânico, que é a estratégia mais simples. Quando algo dá errado, o analisador registra a mensagem com linha, coluna e tamanho do trecho, joga fora a parte com problema e volta ao estado inicial. Assim um erro não esconde os próximos, e o usuário vê todos de uma vez. São cinco tipos de erro:

Erro	Exemplo	O que acontece
Caractere inválido	`idade@`, `#`, `_teste`, acento fora de string	o caractere é descartado
Identificador com mais de 15 caracteres	`nomeMuitoMaiorQueOPermitido`	o identificador inteiro é descartado
Número mal formado	`12abc`, `3.`	o trecho é descartado até o próximo separador
Literal não fechado	`'sem fechar`	o literal termina no fim da linha

Erro	Exemplo	O que acontece
Comentário não fechado	`{` ou `(*` sem fechamento	o erro é marcado onde o comentário abriu, e o comentário vai só até o fim daquela linha; o resto do programa continua sendo analisado

Correção apontada na primeira entrega

Na avaliação da primeira parte, a observação foi que o ponto do final do programa não estava sendo reconhecido como token. O problema aparecia no `erros.pqk`, que tem um comentário `{` sem fechamento na penúltima linha. Quando o analisador chegava ao fim do arquivo ainda dentro do comentário, ele registrava o erro, mas tudo o que vinha depois do `{` já tinha sido consumido como parte do comentário, inclusive o `end.` da última linha. Por isso a lista de tokens terminava sem o `END` e sem o `PONTO`.

A correção foi tratar o comentário sem fechamento do mesmo jeito que já tratávamos o literal sem fechamento. O erro continua sendo marcado onde o comentário abriu, mas o comentário passa a ir só até o fim daquela linha: o analisador volta para essa posição e continua a leitura dali, então o resto do programa é analisado normalmente. Isso vale para os dois comentários de várias linhas, `{ }` e `(* *)`, e está na função `fecharComentarioNaLinha()`. Com a mudança, o `erros.pqk` continua mostrando os mesmos oito erros léxicos, e a lista de tokens agora termina como deveria:

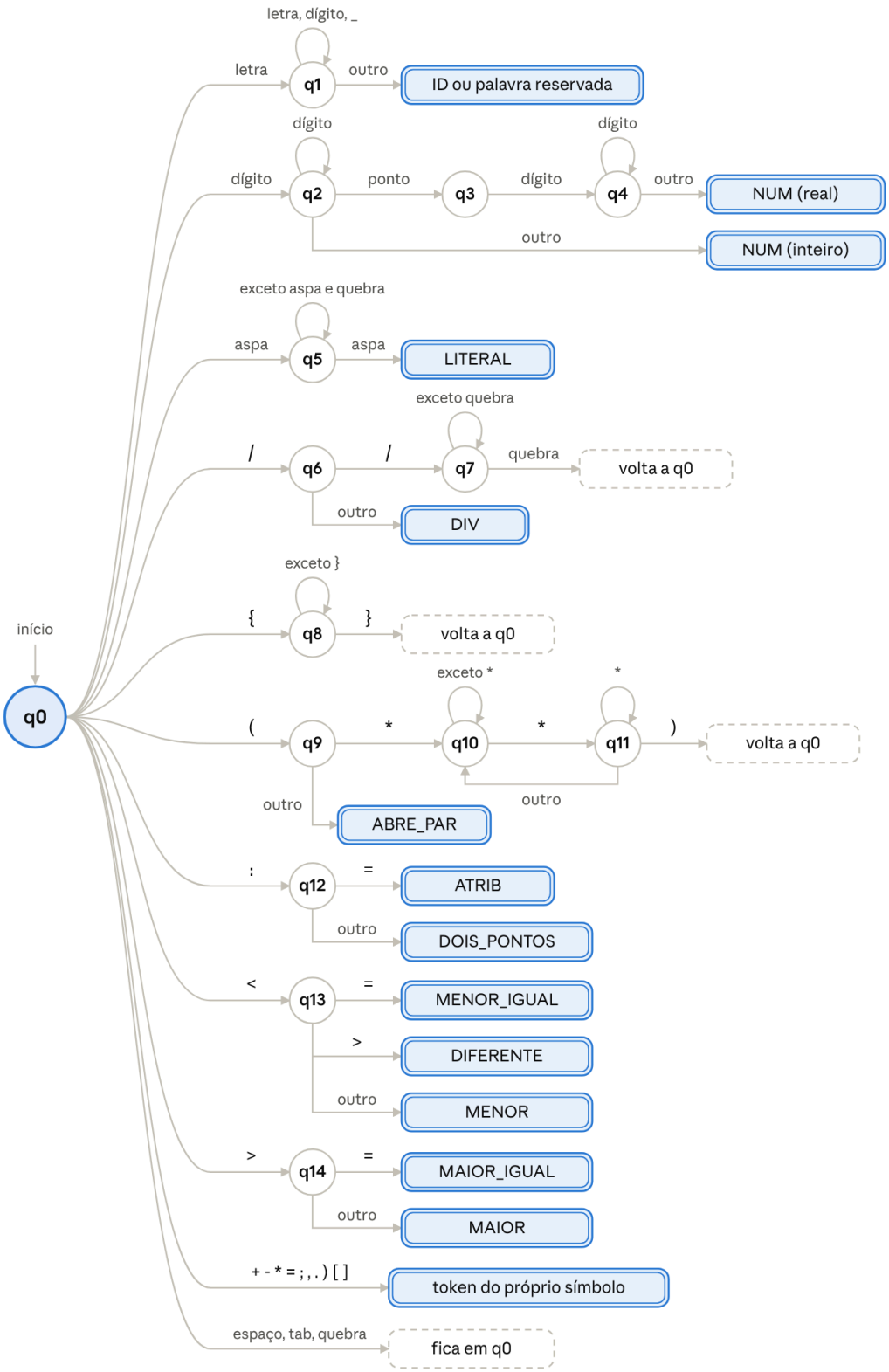
```
Linha 12, coluna 3 [lexico]: Comentario { nao foi fechado
...
13:1      END             end         TS[2]
13:4      PONTO           .           TS[37]
14:1      EOF
```

Autômato finito determinístico

O autômato tem 15 estados, de q0 a q14. No desenho, cada ramo ocupa uma linha: o primeiro caractere do lexema escolhe o ramo, e dali em diante o analisador olha um caractere de cada vez para decidir se continua, se emite o token ou se descarta o que leu.

Autômato do analisador: o primeiro caractere escolhe o ramo

Cada linha corresponde a um ramo do switch em lexer.c



Círculo: estado. Borda dupla: estado final, emite o token. Tracejado: descarta o lexema e volta a q0.
Transição não desenhada: erro léxico, tratado em modo pânico (volta a q0).

AFD do analisador léxico · 15 estados

A palavra "outro" aparece bastante no diagrama. Ela quer dizer qualquer caractere que não continua o lexema atual. Esse caractere não é consumido: o analisador emite o token, volta a q0 e começa o próximo lexema justamente por ele. A tabela abaixo liga cada estado ao nome que ele tem no código.

Estado	Nome em lexer.c	O que significa
q0	`Q0`	início de um novo lexema
q1	`Q_ID`	lendo identificador ou palavra reservada
q2	`Q_NUM`	lendo a parte inteira de um número
q3	`Q_NUM_PONTO`	leu o ponto, espera um dígito
q4	`Q_NUM_REAL`	lendo a parte decimal
q5	`Q_LITERAL`	dentro de um literal
q6	`Q_BARRA`	leu `/`: pode ser divisão ou comentário
q7	`Q_COMENT_LINHA`	dentro de um comentário `//`
q8	`Q_COMENT_CHAVE`	dentro de um comentário `{ }`
q9	`Q_ABRE_PAR`	leu `(`: pode ser parênteses ou comentário
q10	`Q_COMENT_PAR`	dentro de um comentário `(* *)`
q11	`Q_COMENT_PAR_AST`	no comentário `(* *)`, acabou de ler `*`
q12	`Q_DOIS_PONTOS`	leu `:`, pode virar `:=`
q13	`Q_MENOR`	leu `<`, pode virar `<=` ou `<>`
q14	`Q_MAIOR`	leu `>`, pode virar `>=`

Existe ainda um estado de erro no código, o `Q_NUM_ERRO`, que entra em ação quando um número aparece grudado em letras, como em `12abc`. Ele consome o resto da palavra e registra o erro. Deixamos ele fora do desenho para não poluir o diagrama, já que é só um caminho de erro. A correção do comentário também não mudou o desenho: ela só altera o que acontece no caminho de erro dos estados q8, q10 e q11 quando o arquivo acaba.

Tabela de tokens

A tabela a seguir lista os 47 tokens da linguagem. A coluna Valor é o código numérico de cada token, que é simplesmente a posição dele no enum `TipoToken`, em `tokens.h`. A coluna Atributo mostra o que vai junto com o token quando ele é encontrado. Para palavras reservadas, operadores e pontuação é a posição fixa na tabela de símbolos (TS), já que eles são carregados antes da análise. As expressões regulares valem sem diferenciar maiúsculas de minúsculas.

Token	Lexema	Expressão regular	Atributo	Valor
PROGRAM	`program`	`program`	TS[0]	0
BEGIN	`begin`	`begin`	TS[1]	1
END	`end`	`end`	TS[2]	2
CONST	`const`	`const`	TS[3]	3
VAR	`var`	`var`	TS[4]	4

Token	Lexema	Expressão regular	Atributo	Valor
INTEGER	`integer`	`integer`	TS[5]	5
REAL	`real`	`real`	TS[6]	6
CHAR	`char`	`char`	TS[7]	7
STRING	`string`	`string`	TS[8]	8
PROCEDURE	`procedure`	`procedure`	TS[9]	9
FUNCTION	`function`	`function`	TS[10]	10
IF	`if`	`if`	TS[11]	11
THEN	`then`	`then`	TS[12]	12
ELSE	`else`	`else`	TS[13]	13
WHILE	`while`	`while`	TS[14]	14
DO	`do`	`do`	TS[15]	15
REPEAT	`repeat`	`repeat`	TS[16]	16
UNTIL	`until`	`until`	TS[17]	17
BREAK	`break`	`break`	TS[18]	18
CONTINUE	`continue`	`continue`	TS[19]	19
E	`e`	`e`	TS[20]	20
OU	`ou`	`ou`	TS[21]	21
FOR	`for`	`for`	TS[22]	22
TO	`to`	`to`	TS[23]	23
ID	contador, x, Media_1	`[A-Za-z][A-Za-z0-9_]{0,14}`	TS[43] em diante, na ordem em que aparecem	24
NUM	10, 3.14	`[0-9]+(\.[0-9]+)?`	inteiro ou real + valor numérico	25
LITERAL	'Pasquick!'	`'[^'\n]*'`	o próprio texto	26
ATRIB	`:=`	`:=`	TS[24]	27
MAIS	`+`	`\+`	TS[25]	28
MENOS	`-`	`-`	TS[26]	29
MULT	`*`	`\*`	TS[27]	30
DIV	`/`	`/`	TS[28]	31
IGUAL	`=`	`=`	TS[29]	32
DIFERENTE	`<>`	`<>`	TS[30]	33
MENOR	`<`	`<`	TS[31]	34
MENOR_IGUAL	`<=`	`<=`	TS[32]	35
MAIOR	`>`	`>`	TS[33]	36
MAIOR_IGUAL	`>=`	`>=`	TS[34]	37
PONTO_VIRGULA	`;`	`;`	TS[35]	38

Token	Lexema	Expressão regular	Atributo	Valor
VIRGULA	`,`	`,`	TS[36]	39
PONTO	`.`	`\.`	TS[37]	40
DOIS_PONTOS	`:`	`:`	TS[38]	41
ABRE_PAR	`(`	`\(`	TS[39]	42
FECHA_PAR	`)`	`\)`	TS[40]	43
ABRE_COL	`[`	`\[`	TS[41]	44
FECHA_COL	`]`	`\]`	TS[42]	45
EOF	(fim do arquivo)	—	—	46

Espaços e comentários também passam pelo autômato, mas não geram token nenhum. As expressões deles são `[ \t\r\n]+` para os espaços, `//[^\n]*` para o comentário de linha, `\{[^}]*\}` para o comentário com chaves e `\(\*` até `\*\)` para o comentário com parênteses e asterisco.

Analisador sintático

O analisador sintático recebe a lista de tokens e confere se eles formam um programa de acordo com a gramática. Se formam, ele devolve uma árvore sintática que representa o programa. Se não, ele registra os erros e continua, para mostrar todos de uma vez. O código fica em `sintatico.c`, e a estrutura da árvore, em `arvore.c`.

Gramática

Esta é a gramática que o analisador reconhece, escrita em EBNF a partir do Anexo I. Chaves indicam repetição (zero ou mais vezes), colchetes indicam algo opcional e a barra separa alternativas. Os nomes em maiúsculo são tokens. A mesma gramática está no comentário do começo de `sintatico.c`.

```
programa       ::= PROGRAM ID ; declaracoes bloco .

declaracoes    ::= { secao_var | secao_const | procedimento | funcao }
secao_var      ::= VAR item_var { item_var }
item_var       ::= lista_ids : tipo ;
secao_const    ::= CONST item_const { item_const }
item_const     ::= ID [ : tipo ] = expressao ;
lista_ids      ::= ID { , ID }
tipo           ::= ( INTEGER | REAL | CHAR | STRING ) [ '[' NUM ']' ]

procedimento   ::= PROCEDURE ID [ parametros ] ; declaracoes bloco ;
funcao         ::= FUNCTION ID [ parametros ] : tipo ; declaracoes bloco ;
parametros     ::= ( [ grupo { ; grupo } [ ; ] ] )
grupo          ::= lista_ids : tipo

bloco          ::= BEGIN lista_comandos END
lista_comandos ::= comando { ; comando }
comando        ::= atrib_ou_chamada | bloco | cmd_if | cmd_while
                 | cmd_repeat | cmd_for | BREAK | CONTINUE | (vazio)
atrib_ou_chamada ::= ID [ '[' expressao ']' ] := expressao
                   | ID [ argumentos ]
cmd_if         ::= IF expressao THEN comando [ ELSE comando ]
cmd_while      ::= WHILE expressao DO comando
cmd_repeat     ::= REPEAT lista_comandos UNTIL expressao
cmd_for        ::= FOR ID := expressao TO expressao DO comando
argumentos     ::= ( [ expressao { , expressao } ] )

expressao      ::= expr_e { OU expr_e }
expr_e         ::= expr_rel { E expr_rel }
expr_rel       ::= expr_simples [ ( = | <> | < | <= | > | >= ) expr_simples ]
expr_simples   ::= termo { ( + | - ) termo }
termo          ::= fator { ( * | / ) fator }
fator          ::= NUM | LITERAL | ( expressao ) | ( + | - ) fator
                 | ID [ '[' expressao ']' | argumentos ]
```

As expressões foram divididas em seis níveis, um para cada precedência, e é essa divisão que faz a multiplicação acontecer antes da soma sem nenhuma regra extra: quanto mais embaixo na gramática, mais forte o operador.

Nível	Operadores	Regra
1 (menor)	`OU`	expressao
2	`E`	expr_e
3	`= <> < <= > >=`	expr_rel
4	`+ -`	expr_simples
5	`* /`	termo
6 (maior)	`+` e `-` unários, parênteses	fator

Colocamos `E` e `OU` abaixo das comparações, então `x > 1 E x < 5` funciona sem parênteses, e a forma com parênteses do `correto.pqk` continua valendo. As comparações não se encadeiam: `a < b < c` é erro, porque a regra `expr_rel` aceita uma comparação só. O `;` separa comandos, como em Pascal, mas como existe o comando vazio, um `;` antes do `end` também é aceito.

Como o analisador funciona

O analisador é descendente recursivo e preditivo, ou seja, LL(1). Cada regra da gramática virou uma função com o mesmo nome, e cada função chama as funções das regras que aparecem do lado direito. Para decidir qual alternativa seguir, ele olha só o token atual e nunca volta atrás. É por isso que a gramática foi escrita de forma que cada alternativa comece por um token diferente: um comando que começa por `WHILE` só pode ser um while, um que começa por `IF` só pode ser um if, e assim por diante.

Duas funções pequenas fazem quase todo o trabalho repetitivo. A `la()` devolve o tipo do token atual, e a `casar(t)` confere se o token atual é o esperado: se for, avança e devolve a posição dele; se não for, registra o erro "Esperado X, encontrado Y". Com isso, a função de uma regra fica praticamente igual à linha da gramática. A do `while`, por exemplo:

```
static No *comandoWhile(void) {
    No *n = novoNo(NO_WHILE, casar(TK_WHILE));

    n->filho[0] = expressao();
    casar(TK_DO);
    n->filho[1] = comando();
    return n;
}
```

O único lugar em que um token não basta é o comando que começa por identificador, que pode ser uma atribuição ou uma chamada. Resolvemos olhando o token seguinte, depois de consumir o nome: se vier `:=` ou `[`, é atribuição; senão, é chamada de procedimento. O caso clássico do *else* pendurado se resolve sozinho: o `else` sempre fica com o `if` mais próximo, porque é a chamada mais interna de `comandoIf()` que encontra ele primeiro.

Árvore sintática

Em vez de guardar todos os tokens, a árvore guarda só o que importa para as próximas fases. O `;`, o `begin`, o `then` e os parênteses somem, e o que fica é a estrutura do programa. Os nós ficam num vetor fixo, do mesmo jeito que os tokens, então não precisamos de `malloc` nem de

`free`, e a árvore inteira é descartada a cada nova análise. Cada nó guarda o seu tipo (`NO_WHILE`, `NO_ATRIB`, `NO_BINARIO`...), o índice do token de onde veio, para saber a linha e a coluna, até quatro filhos e um ponteiro para o próximo nó da mesma lista, usado nas listas de comandos, declarações e argumentos.

Para o programa de exemplo da seção da linguagem, a árvore impressa pela versão de console fica assim:

```
programa EXEMPLO
  var X : integer
  bloco
    :=
      id X
      num 0
    while
      <
        id X
        num 10
      do
        :=
          id X
          +
            id X
            num 1
```

Dá para ver a precedência funcionando no `x := x + 1`: o `+` fica embaixo do `:=`, com os dois operandos como filhos.

Erros e recuperação

Assim como no léxico, usamos o modo pânico. Quando o analisador encontra um token que não esperava, ele registra o erro e passa a descartar tokens até chegar num token de sincronização, um ponto a partir do qual dá para continuar com segurança: `;`, `end`, `begin`, `until`, o começo de um comando ou o começo de uma declaração.

Só isso ainda gerava erros em cascata, porque depois de um erro o analisador costuma tropeçar de novo antes de se reencontrar. Por isso existe um indicador, o `emPanico`. Ele liga no primeiro erro e, enquanto estiver ligado, nenhuma mensagem nova é registrada. Ele só desliga quando a `casar()` consegue casar um token esperado, ou seja, quando o analisador volta a entender o programa. Na prática, cada erro real gera uma mensagem só.

Para alguns erros comuns, a recuperação é mais esperta do que só descartar tokens, e a mensagem foi escrita pensando em quem está aprendendo a linguagem:

Situação	Exemplo	Mensagem	O que o analisador faz
`;` esquecido entre dois comandos	`x := 10` e, na linha de baixo, `y := 20;`	`Esperado ';' antes de 'y'`	aponta o fim da linha anterior e segue como se o `;` estivesse lá
`;` antes do `else`	`x := y; else`	`'else' inesperado: nao coloque ';' antes do else`	descarta o `else` e analisa o comando dele

Situação	Exemplo	Mensagem	O que o analisador faz
`=` no lugar de `:=`	`nome = 'teste';`	`Use ':=' para atribuicao ('=' e comparacao)`	continua como se fosse uma atribuição
Parêntese não fechado	`y := (x + 3;`	`Esperado ')', encontrado ';'`	sincroniza no `;`
Falta o `do`	`while x < 100` sem `do`	`Esperado 'do', encontrado 'x'`	segue com o comando do laço
Falta o ponto final	`end` sem `.`	`Esperado '.', encontrado fim do arquivo`	—

Quando o erro é de algo que faltou e o próximo token já está em outra linha, a mensagem aponta para logo depois do último token da linha anterior. Sem isso, um `;` esquecido no fim da linha 14 aparecia como erro na linha 15.

Analisador semântico

Muita coisa errada passa pela gramática. Fazer `x := 'texto'` com `x` inteiro, usar uma variável que nunca foi declarada ou chamar uma função com argumentos a menos são programas sintaticamente corretos. O analisador semântico, em `semantico.c`, percorre a árvore montada pelo sintático e confere essas regras. Ao mesmo tempo, ele anota em cada expressão o tipo calculado e liga cada nome à sua declaração, que é o que uma geração de código vai precisar depois.

Como dissemos antes, ele só roda quando o programa não tem erro sintático. Com a árvore incompleta, ele acusaria erros que não existem, como uma variável "não declarada" só porque a declaração dela tinha um erro de sintaxe.

Tabela de declarações e escopos

A tabela de símbolos do léxico tem uma entrada por lexema: o `X` é um só, não importa onde apareça. Isso não serve para o semântico, porque o mesmo nome pode ser uma variável global e, ao mesmo tempo, o parâmetro de uma função. Por isso o semântico tem a sua própria tabela, a de declarações, com uma entrada para cada vez que um nome é declarado, junto com a categoria (variável, constante, parâmetro, procedimento, função ou programa), o tipo, o escopo e a linha.

Os escopos funcionam como uma pilha. Ao entrar numa função ou num procedimento, um escopo novo é aberto, e os parâmetros e as variáveis locais entram nele. Ao sair, essas declarações deixam de ser visíveis, mas continuam na tabela, para aparecer na IDE. Para achar um nome, a busca começa no escopo mais interno e vai para fora, então uma variável local esconde uma global de mesmo nome. Declarar o mesmo nome duas vezes no mesmo escopo é erro.

O nome de uma função entra na tabela antes de o corpo dela ser analisado. É isso que permite a recursão, como no `fatorial` do `completo.pqk`. Para o `correto.pqk`, a tabela fica assim (sem as quatro primeiras linhas, que são `write`, `writeln`, `read` e `readln`):

Nome	Categoria	Tipo	Escopo	Linha	Usada
EXEMPLO	programa	—	global	2	sim
X	variável	integer	global	4	sim
Y	variável	integer	global	4	sim
MEDIA	variável	real	global	5	sim
NOME	variável	string	global	6	sim
LIMITE	constante	integer	global	8	sim
PI	constante	real	global	9	não
DOBRO	função	integer	global	11	sim
N	parâmetro	integer	DOBRO	11	sim

A coluna Usada não gera erro, mas ajuda a achar declarações esquecidas, como a constante `PI`, que é declarada e nunca usada.

Tipos

Os tipos da linguagem são `integer`, `real`, `char` e `string`, além dos vetores. Existe ainda um tipo interno, que chamamos de lógico: é o resultado de uma comparação ou de `E` e `OU`. Como a gramática não tem o tipo `boolean`, não dá para declarar uma variável lógica, e esse tipo só aparece nas condições do `if`, do `while` e do `until`.

Um literal com um caractere só, como `'a'`, é do tipo `char`, e os demais são `string`. Como um `char` pode ser usado onde se espera uma `string`, o `'a'` serve para os dois. As regras dos operadores são estas:

Operação	Operandos aceitos	Resultado
`+ - * /`	números (integer e real)	integer se os dois forem integer, senão real
`+`	textos (char e string)	string (concatenação)
`= <> < <= > >=`	dois números, ou dois textos	lógico
`E OU`	dois lógicos	lógico
`-` e `+` unários	um número	o mesmo tipo do número

Como a linguagem não tem `div`, `integer / integer` dá `integer`, como em C. Se um dos lados for `real`, o resultado é `real`. Na atribuição e na passagem de argumentos, o valor precisa caber no destino: um inteiro pode ir para uma variável real, mas o contrário perderia a parte decimal e é erro.

Destino	Aceita
integer	integer
real	real e integer
char	char
string	string e char
vetor	vetor do mesmo tipo e do mesmo tamanho

Regras verificadas

Cada regra abaixo tem pelo menos um caso no `erros_semanticos.pqk`. As mensagens são as que o compilador mostra.

Regra	Exemplo de mensagem
Todo nome precisa ter sido declarado antes do uso	`'z' nao foi declarado`
Sem nome repetido no mesmo escopo	`'x' ja foi declarado neste escopo (linha 7)`
Atribuição com tipos compatíveis	`Tipos incompativeis: 'x' e integer, mas a expressao e real`

Regra	Exemplo de mensagem
Operadores só com os tipos que eles aceitam	`O operador '+' nao se aplica a string e integer`
Condição de if, while e until precisa ser lógica	`A condicao do 'if' precisa ser uma comparacao, mas e do tipo integer`
E e OU só entre comparações	`O operador 'E' precisa de comparacoes dos dois lados, mas recebeu logico e string`
Constante não recebe valor, e o valor dela tem que bater com o tipo	`'LIMITE' e constante e nao pode receber valor`
Chamada com a quantidade certa de argumentos	`'imprimir' espera 1 argumento(s), mas recebeu 2`
Cada argumento compatível com o parâmetro	`Argumento 1 de 'imprimir': esperado integer, encontrado string`
Procedimento não devolve valor, então não entra em expressão	`'imprimir' e um procedimento: nao devolve valor e nao pode ser usado em expressao`
Só se chama procedimento ou função	`'x' e variavel, nao procedimento nem funcao`
O nome da função só recebe valor dentro dela	`So da para atribuir a 'dobro' dentro da propria funcao (valor de retorno)`
Toda função precisa receber um valor de retorno	`A funcao 'dobro' nunca recebe valor de retorno (falta 'dobro := ...')`
Índice só em vetor ou string, e sempre integer	`O indice de 'w' deve ser integer, mas e real`
Tamanho de vetor inteiro e maior que zero	`O tamanho do vetor 'v' deve ser um inteiro maior que zero`
Variável e limites do for são integer	`A variavel de controle do for precisa ser integer, mas 'media' e real`
break e continue só dentro de laço	`'break' so pode ser usado dentro de while, repeat ou for`
read só lê variáveis; vetor inteiro não é lido nem escrito	`Argumento 1 de 'read' precisa ser uma variavel`

A árvore com os tipos

Depois do semântico, a mesma árvore do exemplo ganha o tipo de cada expressão:

```
programa EXEMPLO
  var X : integer
  bloco
    :=
      id X   <integer>
      num 0   <integer>
    while
      <   <logico>
        id X   <integer>
        num 10   <integer>
      do
        :=
          id X   <integer>
          +   <integer>
            id X   <integer>
            num 1   <integer>
```

A estrutura é a mesma da seção anterior. O que muda é que a comparação `x < 10` virou lógica, e é por isso que ela pode ser a condição do `while`. Se no lugar dela estivesse só `x`, o tipo seria `integer` e o semântico acusaria o erro.

A IDE Chronus

A Chronus continua sendo uma janela só. Em cima ficam os botões Novo, Salvar, Compilar e, agora, Fase. No meio fica o editor, embaixo um painel com os resultados e, no rodapé, uma barra de status.

A parte de que mais gostamos é a análise enquanto o usuário digita. Toda vez que o texto muda, o programa inteiro passa de novo pelas três fases. Pode parecer exagero, mas para programas desse tamanho isso continua levando muito menos que um quadro de tela, então o erro aparece no momento em que é digitado e some quando é corrigido.

Os erros aparecem de várias formas ao mesmo tempo. O trecho ganha um sublinhado ondulado vermelho, parecido com o do corretor ortográfico, e o número da linha fica vermelho. Passando o mouse por cima, aparece a mensagem com a fase, a linha e a coluna, como em "Erro semântico (lin 26, col 8)". Nos erros semânticos, o sublinhado cobre a expressão inteira que está errada, e não só o primeiro token dela. A barra de status também fica vermelha e mostra o erro da linha onde está o cursor. Os trechos com erro léxico continuam pintados de vermelho claro, porque não viram token; os erros sintáticos e semânticos caem sobre tokens válidos e só ganham o sublinhado.

Como os tokens já estavam prontos, colorir o código saiu quase de graça: palavras reservadas em azul, números em verde-claro, literais em laranja e operadores em amarelo. Comentários não viram token e ficam na cor padrão, verde.

O painel de baixo agora tem cinco abas: Tokens, Símbolos, Árvore, Declarações e Erros. A aba Árvore mostra a árvore sintática, com o tipo de cada expressão em outra cor; a de Declarações mostra a tabela do semântico; e a de Erros mostra a posição, a fase e a mensagem. Clicar numa linha de qualquer aba leva o cursor até aquele ponto do código, o que ajuda bastante quando o programa é maior que a tela.

Para usar, F5 (ou o botão Compilar) roda a análise e abre a aba de erros, F6 (ou o botão Fase) troca a fase, Ctrl+S salva e Ctrl+V cola. Para abrir um arquivo, basta arrastar o `.pqk` para dentro da janela ou passar o caminho ao abrir o programa, como em `chronus.exe exemplos\completo.pqk`. Continuamos deixando de fora seleção de texto, copiar e desfazer, para o editor continuar simples.

Mostrando o compilador por partes

Como o trabalho foi feito em etapas, quisemos poder mostrar cada fase sozinha. O botão Fase, ou a tecla F6, escolhe até onde a análise vai: só o léxico, o léxico e o sintático, ou as três fases. Ao trocar, a IDE já abre a aba que mostra o resultado daquela fase: Tokens para o léxico, Árvore para o sintático e Declarações para o semântico. A fase atual aparece também na barra de status.

O teste de que mais gostamos para mostrar a diferença entre as fases é abrir o `erros_semanticos.pqk` com a fase sintática: nenhum erro, porque a sintaxe do programa está toda certa. Ao trocar para a fase semântica, aparecem 25 erros.

Versão de console

A versão de console, que agora se chama `pasquick.exe`, usa exatamente o mesmo código das análises. Ela recebe o arquivo e algumas opções:

Opção	O que faz
`-f lexico|sintatico|semantico`	até qual fase compilar (o padrão é a semântica, o compilador inteiro)
`-t`	lista de tokens
`-s`	tabela de símbolos do léxico
`-a`	árvore sintática, com os tipos quando o semântico roda
`-d`	tabela de declarações do semântico

Sem as opções `-t`, `-s`, `-a` e `-d`, ela mostra tudo. Os erros aparecem sempre, com linha, coluna e fase, e o programa termina com código 0 se o fonte está correto e 2 se tem erro, o que facilita usar em scripts de teste. Para compilar, o `build.bat` gera a IDE (`build\chronus.exe`) e o `build_console.bat` gera a versão de console. Os dois usam o gcc que vem com o instalador da raylib.

Testes

Os programas de teste ficam na pasta `exemplos/`, e rodamos todos tanto na versão de console quanto na IDE.

Arquivo	O que testa	Resultado
`correto.pqk`	o programa da primeira entrega, com um pouco de tudo	109 tokens, 52 símbolos, 13 declarações e nenhum erro
`completo.pqk`	todas as construções: for, repeat, break, continue, vetores, constantes com e sem tipo, função recursiva, procedimento e concatenação	292 tokens, 25 declarações e nenhum erro
`erros.pqk`	um erro léxico de cada tipo	os 8 erros léxicos, nas mesmas posições da primeira entrega, e o `end.` final reconhecido
`erros_sintaticos.pqk`	erros de sintaxe comuns	9 erros sintáticos numa única análise
`erros_semanticos.pqk`	um erro de cada regra semântica, num programa sintaticamente correto	nenhum erro até a fase sintática; 25 erros na fase semântica

No `erros.pqk`, a compilação completa mostra também 5 erros sintáticos. Eles são consequência dos trechos que o léxico descartou: depois que o `12abc` é jogado fora, por exemplo, sobra um `idade := ;`, que é mesmo um erro de sintaxe. Com a fase léxica selecionada, aparecem só os 8 erros léxicos.

O `erros_sintaticos.pqk` foi escrito para testar a recuperação. Os nove erros são encontrados numa única análise, cada um com uma mensagem só:

Linha:coluna	Trecho	Mensagem
5:7	`z : ;`	`Esperado um tipo (integer, real, char ou string), encontrado ';'`
14:10	`x := 10`	`Esperado ';' antes de 'y'`
18:3	`else`	`'else' inesperado: nao coloque ';' antes do else`
20:16	`while x < 100`	`Esperado 'do', encontrado 'x'`
22:8	`nome = 'teste';`	`Use ':=' para atribuicao ('=' e comparacao)`
23:14	`y := (x + 3;`	`Esperado ')', encontrado ';'`
24:17	`x := soma(x, y;`	`Esperado ')', encontrado ';'`
27:9	`until ;`	`Esperado uma expressao, encontrado ';'`
28:4	`end`	`Esperado '.', encontrado fim do arquivo`

O `erros_semanticos.pqk` tem um erro de cada regra semântica, um por linha, num programa que passa limpo pelo sintático. Alguns dos 25 erros encontrados:

Linha:coluna	Trecho	Mensagem
5:22	`ERRADA : integer = 2.5;`	`A constante 'ERRADA' e integer, mas o valor e real`
8:3	`x : real;`	`'x' ja foi declarado neste escopo (linha 7)`
20:10	`function dobro(...)`	`A funcao 'dobro' nunca recebe valor de retorno (falta 'dobro := ...')`
26:8	`x := 3.7;`	`Tipos incompativeis: 'x' e integer, mas a expressao e real`
27:3	`LIMITE := 20;`	`'LIMITE' e constante e nao pode receber valor`
28:3	`z := 1;`	`'z' nao foi declarado`
32:6	`if x then`	`A condicao do 'if' precisa ser uma comparacao, mas e do tipo integer`
36:3	`imprimir(1, 2);`	`'imprimir' espera 1 argumento(s), mas recebeu 2`
38:8	`x := imprimir(5);`	`'imprimir' e um procedimento: nao devolve valor e nao pode ser usado em expressao`
46:3	`break;`	`'break' so pode ser usado dentro de while, repeat ou for`

Por último, queríamos ter certeza de que nenhuma entrada estranha travasse o compilador, já que na IDE ele roda a cada tecla, com o programa quase sempre pela metade. Escrevemos um script que pega os exemplos e estraga eles ao acaso, apagando, trocando e inserindo tokens, e rodamos o compilador em 2.300 versões assim, compilado com o AddressSanitizer do gcc, que

acusa qualquer acesso inválido de memória. Nenhuma delas travou, entrou em laço infinito ou acessou memória indevida. Testamos também arquivo vazio, arquivo só com `program` e bytes aleatórios.

Conclusão

Com as três fases de análise prontas, o compilador cobre toda a parte de entender o programa: ele separa o texto em tokens, confere a estrutura pela gramática e verifica se o programa faz sentido, com escopos e tipos. Os erros de qualquer fase aparecem com linha e coluna enquanto o programa é escrito, e a análise continua depois de cada um.

O que mais ficou para nós desta segunda parte foi ver como as fases se apoiam umas nas outras. A tabela de símbolos do léxico, a árvore do sintático e a tabela de declarações do semântico são três visões do mesmo programa, cada uma mais perto do significado do que a anterior. E, de novo, a teoria virou código quase sem adaptação: assim como cada estado do autômato tinha virado um `case`, cada regra da gramática virou uma função. Quando precisávamos mudar alguma coisa na linguagem, bastava mudar a regra no papel e a função correspondente.

Também ficou claro que a recuperação de erros é a parte que mais dá trabalho em cada fase. Reconhecer um programa correto é relativamente direto. O difícil é continuar de um jeito útil depois de um erro, sem esconder os próximos e sem inventar erros que não existem. A correção do comentário sem fechamento, apontada na primeira avaliação, foi um exemplo disso: o erro era detectado, mas a recuperação apagava o resto do programa.

O próximo passo natural é a geração de código. A árvore já sai do semântico com os tipos anotados e com cada nome ligado à sua declaração, que é exatamente o que um gerador de código ou um interpretador precisa para começar.